

kdy40929's Reference List v1.0 - 2026.01.12

1 Math	2	3.3.4 MCMF	11
1.1 Linear Algebra	2	3.4 Trees	12
1.1.1 Matrix Operations	2	3.4.1 LCA	12
1.1.2 Gauss Elimination	2	3.4.2 HLD	12
1.1.3 XOR basis	2	4 DP	13
1.2 Number Theory	2	4.1 Basic DP	13
1.2.1 Euler Phi	2	4.1.1 LIS	13
1.2.2 Miller labin & Pollard Rho	2	4.2 DP optimization	13
1.2.3 Combination with Lucas Theorem	3	4.2.1 Convex Hull Trick	13
1.3 Geometry	3	5 String	13
1.3.1 Geometry	3	5.1 Pattern Matching	13
1.4 Others	4	5.1.1 KMP	13
1.4.1 FFT	4	5.1.2 Aho Corasick	13
1.4.2 Floor Sum	4	5.2 Substring Algorithm	14
2 Data Structures	4	5.2.1 Rolling Hash	14
2.1 Deque	4	5.2.2 Z	14
2.1.1 Deque Trick	4	5.2.3 Manacher	14
2.2 Segment Tree	4	5.2.4 Suffix array & LCP array	14
2.2.1 Fenwick Tree	4	6 Misc	15
2.2.2 Non-recurisve Range Update Range Query	5	6.1 Useful Codes	15
2.2.3 Merge Sort Tree	5	6.1.1 Fast I/O	15
2.2.4 2D Segment Tree	6	6.1.2 Coordinate Compression	15
2.2.5 Maximum Subarray Sum Segment Tree	6	6.2 Others	15
2.3 Trie	7	6.2.1 Graph Theorem	15
2.3.1 1D Trie	7	6.2.2 Useful Number	15
3 Graphs	8	6.2.3 Useful Sequences	16
3.1 Shortest Path	8		
3.1.1 Dijkstra	8		
3.1.2 Bellman Ford	8		
3.1.3 SPFA	8		
3.1.4 Floyd Warshall	8		
3.2 SCC and similar	8		
3.2.1 SCC	8		
3.2.2 2-SAT	9		
3.2.3 Articulation	9		
3.3 Matching & Flow	10		
3.3.1 Bipartite Matching - Hopcroft Karp	10		
3.3.2 Dinic Algorithm	10		
3.3.3 Dinic with low memory	11		

1. Math

1.1. Linear Algebra

1.1.1 Matrix Operations

```
def zero(n):
    return [[0]*n for _ in range(n)]

def eye(n):
    E = [[0]*n for _ in range(n)]
    for i in range(n):
        E[i][i] = 1
    return E

def add(A, B, mod):
    C = [[0 for _ in range(len(A[0]))] for _ in range(len(A))]
    for i in range(len(A)):
        for j in range(len(A[0])):
            C[i][j] = (A[i][j] + B[i][j]) % mod
    return C

def multiply(A, B, mod):
    C = [[0 for _ in range(len(B[0]))] for _ in range(len(A))]
    for i in range(len(A)):
        for j in range(len(B[0])):
            for k in range(len(B)):
                C[i][j] += A[i][k] * B[k][j]
            C[i][j] %= mod
    return C

def power(mat, exp, mod):
    n = len(mat)
    ans = eye(n)
    base = mat
    while exp:
        if exp&1: ans = multiply(ans, base, mod)
        base = multiply(base, base, mod)
        exp >>= 1
    return ans

def powsum(mat, exp, mod):
    # mat^exp, eye + mat + ... + mat^(exp-1)
    n = len(mat)
    m1, m2 = eye(n), zero(n)
    base1, base2 = [x[:] for x in mat], eye(n)
    while exp:
        if exp&1:
            m2 = add(m2, multiply(m1, base2, mod), mod)
            m1 = multiply(m1, base1, mod)
            base2 = multiply(base2, add(eye(n), base1, mod), mod)
            base1 = multiply(base1, base1, mod)
            exp >>= 1
    return (m1, m2)
```

1.1.2 Gauss Elimination

BOJ 22940

```
n = int(input())
eq = [[*map(int, input().split())] for _ in range(n)]
for k in range(n):
    for i in range(k+1, n):
        for j in range(n, k-1, -1):
            eq[i][j] = eq[k][k] * eq[i][j] - eq[i][k] * eq[k][j]
ans = [0]*n
for i in range(n-1, -1, -1):
    tmp = eq[i][n]
    for j in range(i+1, n):
        tmp -= eq[i][j] * ans[j]
    ans[i] = tmp//eq[i][i]
print(*ans)
```

1.1.3 XOR basis

```
def make_xor_basis(arr):
    n = len(arr)
    basis = [0]*60
    for i in range(n):
        for j in range(59, -1, -1):
            if (arr[i]>>j) & 1:
                if basis[j]: arr[i] ^= basis[j]
                else: basis[j] = arr[i]; break
    return basis
```

1.2. Number Theory

1.2.1 Euler Phi

```
def euler_phi(n):
    if n < 4: return n - 1
    ans = n
    p = 2
    while p*p <= n:
        if n % p == 0:
            ans = ans // p * (p - 1)
            while n % p == 0:
                n //= p
        p += 1
    if n > 1:
        ans = ans // n * (n - 1)
    return ans
```

1.2.2 Miller labin & Pollard Rho

```
plist = [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37]
def isprime(k):
    if k <= 37 and k in plist: return 1
    elif k <= 37: return 0
    tmp = k-1
    cnt = 0
    while tmp%2 == 0:
        tmp //= 2
        cnt += 1
    s, t = cnt, tmp
    for p in plist:
        tmp = pow(p, t, k)
        if tmp == 1: continue
```

```

    flag = 0
    for _ in range(s):
        if tmp == k-1:
            flag = 1; break
        tmp = pow(tmp, 2, k)
    if flag == 0: return 0
    return 1

def f(x,n):
    return (x*x+1)%n

def polar(n, x):
    tmp = x
    if isprime(n): return n
    for i in plist:
        if n%i == 0: return i
    y = x
    gcd = 1
    while gcd == 1:
        x = f(x, n)
        y = f(f(y, n), n)
        gcd = math.gcd(abs(x-y), n)
    if gcd == n:
        return polar(n, tmp+1)
    if isprime(gcd):
        return gcd
    return polar(gcd, 2)

```

1.2.3 Combination with Lucas Theorem

```

def combination(n, k, p):
    if k < 0 or k > n: return 0
    numer, denom = 1, 1
    if 2*k > n: k = n-k
    for i in range(n, n-k, -1):
        numer = (numer * i) % p
    for j in range(1, k+1):
        denom = (denom * j) % p
    return numer * pow(denom, -1, p) % p

def lucas_comb(n, k, p):
    ans = 1
    while n > 0 or k > 0:
        ans = ans * combination(n%m, k%m, m) % m
        n //= m; k //= m
    return ans

```

1.3. Geometry

1.3.1 Geometry

```

from functools import cmp_to_key

def cross(p1, p2, p3, p4):
    # p1 -> p2 / p3 -> p4
    return (p2[0]-p1[0])*(p4[1]-p3[1]) - (p4[0]-p3[0])*(p2[1]-p1[1])

def di_area(pts):

```

```

    s = 0
    for i in range(len(pts)):
        s += pts[i][0]*pts[i-1][1] - pts[i-1][0]*pts[i][1]
    return abs(s)

def intersect(p1, p2, q1, q2):
    # p1 - p2 / q1 - q2
    a = cross(p1, p2, p1, q1) * cross(p1, p2, p1, q2)
    b = cross(q1, q2, q1, p1) * cross(q1, q2, q1, p2)
    if a == b == 0:
        return max(p1, p2) >= min(q1, q2) and max(q1, q2) >= min(p1, p2)
    return a <= 0 and b <= 0

def dist(p1, p2):
    return (p1[0] - p2[0])**2 + (p1[1] - p2[1])**2

def comp(base, p1, p2):
    cp = cross(base, p1, base, p2)
    if cp > 0: return -1
    if cp < 0: return 1
    return 1 if dist(base, p1) > dist(base, p2) else -1

def convex_hull(points):
    base = min(points, key = lambda x: (x[1], x[0]))
    sorted_points = sorted(points, key=cmp_to_key(lambda p1, p2: comp(base, p1, p2)))
    hull = []
    for p in sorted_points:
        while len(hull) >= 2 and cross(hull[-2], hull[-1], hull[-2], p) <= 0:
            hull.pop()
        hull.append(p)
    return hull

def check_inconvex(pg, point, kind):
    # include edge if kind == 1
    if kind == 1:
        if cross(pg[0], pg[1], pg[0], point) < 0: return 0
        if cross(pg[0], pg[-1], pg[0], point) > 0: return 0
        left, right = 0, len(pg) - 1
        while left + 1 < right:
            mid = (left + right)//2
            if cross(pg[0], pg[mid], pg[0], point) >= 0:
                left = mid
            else: right = mid
        return cross(pg[left], point, point, pg[right]) <= 0
    else:
        if cross(pg[0], pg[1], pg[0], point) <= 0: return 0
        if cross(pg[0], pg[-1], pg[0], point) >= 0: return 0
        left, right = 0, len(pg) - 1
        while left + 1 < right:
            mid = (left + right)//2
            if cross(pg[0], pg[mid], pg[0], point) > 0:
                left = mid
            else: right = mid
        return cross(pg[left], point, point, pg[right]) < 0

def rot_calipers(hull):

```

```

# find a maximum distance among points of hull
maxdst = 0
j = 1
n = len(hull)
p1, p2 = hull[0], hull[1]
for i in range(n):
    while j < 2*n and cross(hull[i], hull[(i+1)%n], hull[j%n], hull[(j+1)%n]) >= 0:
        tmp = dist(hull[i], hull[j%n])
        if tmp > maxdst:
            maxdst = tmp
            p1, p2 = hull[i], hull[j%n]
        j += 1
    tmp = dist(hull[i], hull[j%n])
    if tmp > maxdst:
        maxdst = tmp
        p1, p2 = hull[i], hull[j%n]
return [p1, p2]

```

1.4. Others

1.4.1 FFT

```

mod = 998244353
g = 3

```

```

def ntt(a, inverse):
    n = len(a)
    j = 0
    for i in range(1, n):
        bit = n >> 1
        while j >= bit:
            j -= bit
            bit >>= 1
        j += bit
        if i < j: a[i], a[j] = a[j], a[i]
    step = 2
    while step <= n:
        half = step//2
        u = pow(g, (mod-1)//step, mod)
        if inverse: u = pow(u, -1, mod)
        for i in range(0, n, step):
            w = 1
            for j in range(i, i+half):
                tmp = a[j+half] * w % mod
                a[j+half] = (a[j] - tmp) % mod
                a[j] = (a[j] + tmp) % mod
                w = w * u % mod
            step *= 2
    if inverse:
        inv = pow(n, -1, mod)
        for i in range(n):
            a[i] = a[i] * inv % mod

def multiply(a, b):
    deg = len(a) + len(b) - 1
    n = 1 << deg.bit_length()
    a.extend([0 for _ in range(n-len(a))])

```

```

b.extend([0 for _ in range(n-len(b))])
ntt(a, False)
ntt(b, False)
for i in range(n):
    a[i] = a[i] * b[i] % mod
ntt(a, True)
return a[:deg]

```

1.4.2 Floor Sum

```

def floor_sum(a, b, c, n):
    # sum of [(ax+b)/c] for x = 0 to x = n
    if a == 0: return (b//c)*(n+1)
    if a >= c or b >= c:
        return n*(n+1)//2*(a//c) + (n+1)*(b//c) + floor_sum(a%c, b%c, c, n)
    m = (a*n+b)//c
    return m*n - floor_sum(c, c-b-1, a, m-1)

```

2. Data Structures

2.1. Deque

2.1.1 Deque Trick

```

from collections import deque

n, l = map(int, input().split())
arr = [*map(int, input().split())]
Q = deque()
for i in range(n):
    while Q and Q[-1][0] >= arr[i]:
        Q.pop()
    Q.append((arr[i], i))
    if Q[0][1] <= i-1:
        Q.popleft()
    print(Q[0][0], end=' ')

```

2.2. Segment Tree

2.2.1 Fenwick Tree

```

class Fenwick:
    def __init__(self, n):
        self.n = n
        self.bit = [0]*(n+1)

    def update(self, idx, val):
        while idx <= self.n:
            self.bit[idx] += val
            idx += idx & -idx

    def count(self, idx): # sum [1..idx]
        s = 0
        while idx > 0:
            s += self.bit[idx]
            idx -= idx & -idx
        return s

    def kth(self, k):

```

```

# smallest idx with prefix sum >= k
if k <= 0: return 0
if self.count(self.n) < k: return self.n + 1
idx = 0
step = 1 << (self.n.bit_length() - 1)
while step:
    nxt = idx + step
    if nxt <= self.n and self.bit[nxt] < k:
        k -= self.bit[nxt]
        idx = nxt
    step >>= 1
return idx + 1

```

2.2.2 Non-recursive Range Update Range Query

mod = 998244353

```

class LazySeg():
    def __init__(self, arr):
        self.n = len(arr)
        self.size = 1 << ((self.n-1).bit_length())
        self.h = self.size.bit_length() - 1
        self.seg = [0]*(2*self.size)
        self.lazya = [1]*self.size
        self.lazyb = [0]*self.size
        base = self.size
        for i, v in enumerate(arr):
            self.seg[base + i] = v % mod
        for i in range(self.size-1, 0, -1):
            self.seg[i] = (self.seg[2*i] + self.seg[2*i+1]) % mod

    def _apply(self, i, a, b, leng):
        self.seg[i] = (a*self.seg[i] + b*leng) % mod
        if i < self.size:
            self.lazya[i] = (a * self.lazya[i]) % mod
            self.lazyb[i] = (a * self.lazyb[i] + b) % mod

    def _push(self, idx):
        for s in range(self.h, 0, -1):
            i = idx >> s
            a, b = self.lazya[i], self.lazyb[i]
            if a != 1 or b != 0:
                tmp = 1 << (s-1)
                self._apply(2*i, a, b, tmp)
                self._apply(2*i+1, a, b, tmp)
                self.lazya[i] = 1
                self.lazyb[i] = 0

        return

    def _pull(self, idx):
        leng = 1
        while idx > 1:
            idx >>= 1
            leng <<= 1
            self.seg[idx] = (self.seg[2*idx] + self.seg[2*idx+1]) % mod
            a, b = self.lazya[idx], self.lazyb[idx]
            if a != 1 or b != 0:

```

```

self.seg[idx] = (a*self.seg[idx] + b * leng) % mod

```

```

def update(self, left, right, a, b):
    # [left, right] update
    if left > right: return
    a &= mask; b &= mask
    l0, r0 = left + self.size, right + 1 + self.size
    self._push(l0)
    self._push(r0-1)
    leng = 1
    l, r = l0, r0
    while l < r:
        if l&1:
            self._apply(l, a, b, leng)
            l += 1
        if r&1:
            r -= 1
            self._apply(r, a, b, leng)
        l >>= 1
        r >>= 1
        leng <<= 1
    self._pull(l0)
    self._pull(r0-1)
    return

```

```

def range_sum(self, left, right):
    # [left, right] sum
    if left > right: return 0
    l, r = left + self.size, right + 1 + self.size
    self._push(l)
    self._push(r-1)
    lsum, rsum = 0, 0
    while l < r:
        if l&1:
            lsum = (lsum + self.seg[l]) % mod
            l += 1
        if r&1:
            r -= 1
            rsum = (rsum + self.seg[r]) % mod
        l >>= 1
        r >>= 1
    return (lsum + rsum) % mod

```

2.2.3 Merge Sort Tree

```

from bisect import bisect_right
def merge(l1, l2):
    i, j = 0, 0
    ans = []
    while i < len(l1) and j < len(l2):
        if l1[i] < l2[j]:
            ans.append(l1[i])
            i += 1
        else:
            ans.append(l2[j])
            j += 1

```

```

    if i == len(l1):
        while j < len(l2):
            ans.append(l2[j])
            j += 1
    elif j == len(l2):
        while i < len(l1):
            ans.append(l1[i])
            i += 1
    return ans

def makeseg(left, right, i):
    if left == right:
        mstree[i].append(arr[left])
        return
    mid = (left + right)//2
    makeseg(left, mid, 2*i)
    makeseg(mid+1, right, 2*i+1)
    mstree[i] = merge(mstree[2*i], mstree[2*i+1])
    return

def count(left, right, i, start, end, val):
    # the number of elements bigger than val in [left, right]
    if end < left or right < start: return 0
    if start <= left and right <= end:
        return right - left + 1 - bisect_right(mstree[i], val)
    mid = (left + right)//2
    return count(left, mid, 2*i, start, end, val) + count(mid+1, right, 2*i+1, start,
        ↪ end, val)

```

2.2.4 2D Segment Tree

```

import sys
input = sys.stdin.readline
def makekey(lx, rx, ix, ly, ry, iy):
    if ly == ry:
        if lx == rx:
            seg[ix][iy] = board[lx][ly]
        else:
            seg[ix][iy] = seg[2*ix][iy] + seg[2*ix+1][iy]
    return
    my = (ly + ry)//2
    makekey(lx, rx, ix, ly, my, 2*iy)
    makekey(lx, rx, ix, my+1, ry, 2*iy+1)
    seg[ix][iy] = seg[ix][2*iy] + seg[ix][2*iy+1]
    return

def makex(lx, rx, ix):
    if lx == rx:
        makekey(lx, rx, ix, 0, n-1, 1)
        return
    mx = (lx + rx)//2
    makex(lx, mx, 2*ix)
    makex(mx+1, rx, 2*ix+1)
    makekey(lx, rx, ix, 0, n-1, 1)
    return

def updatex(lx, rx, ix, ly, ry, iy, x, y, val):

```

```

    if y > ry or y < ly: return
    if ly == ry:
        if lx == rx:
            seg[ix][iy] = val
        else:
            seg[ix][iy] = seg[2*ix][iy] + seg[2*ix+1][iy]
    return
    my = (ly + ry)//2
    updatex(lx, rx, ix, ly, my, 2*iy, x, y, val)
    updatex(lx, rx, ix, my+1, ry, 2*iy+1, x, y, val)
    seg[ix][iy] = seg[ix][2*iy] + seg[ix][2*iy+1]
    return

def updatex(lx, rx, ix, x, y, val):
    if x < lx or x > rx: return
    if lx == rx:
        updatex(lx, rx, ix, 0, n-1, 1, x, y, val)
        return
    mx = (lx + rx)//2
    updatex(lx, mx, 2*ix, x, y, val)
    updatex(mx+1, rx, 2*ix+1, x, y, val)
    updatex(lx, rx, ix, 0, n-1, 1, x, y, val)

def getsumy(ix, ly, ry, iy, y1, y2):
    if y2 < ly or ry < y1: return 0
    if y1 <= ly and ry <= y2: return seg[ix][iy]
    my = (ly + ry)//2
    return getsumy(ix, ly, my, 2*iy, y1, y2) + getsumy(ix, my+1, ry, 2*iy+1, y1, y2)

def getsumx(lx, rx, ix, x1, x2, y1, y2):
    if x2 < lx or rx < x1: return 0
    if x1 <= lx and rx <= x2:
        return getsumy(ix, 0, n-1, 1, y1, y2)
    mx = (lx + rx)//2
    return getsumx(lx, mx, 2*ix, x1, x2, y1, y2) + getsumx(mx+1, rx, 2*ix+1, x1, x2,
        ↪ y1, y2)

```

```

n, m = map(int, input().split())
board = [[*map(int, input().split())] for _ in range(n)]
size = 2<<n.bit_length()
seg = [[0] * size for _ in range(size)]
makex(0, n-1, 1)
for _ in range(m):
    query = [*map(int, input().split())]
    if query[0] == 0:
        x, y, c = query[1:]
        updatex(0, n-1, 1, x-1, y-1, c)
    else:
        x1, y1, x2, y2 = map(lambda x: int(x)-1, query[1:])
        print(getsumx(0, n-1, 1, x1, x2, y1, y2))

```

2.2.5 Maximum Subarray Sum Segment Tree

```

class Node:
    def __init__(self, w=0):
        self.l = self.r = self.m = max(0, w)
        self.s = w

```

```

def merge(n1, n2):
    n3 = Node()
    n3.l = max(n1.l, n1.s + n2.l)
    n3.r = max(n2.r, n2.s + n1.r)
    n3.m = max(n1.m, n2.m, n1.r + n2.l)
    n3.s = n1.s + n2.s
    return n3

def rev(n1):
    n1.l, n1.r = n1.r, n1.l
    return n1

class MineSeg:
    def __init__(self, arr):
        k = len(arr) - 1
        self.h = k.bit_length()
        self.size = 1<<self.h
        self.seg = [Node(0) for _ in range(2*self.size)]
        self.lazy = [-INF]*(self.size)
        for i in range(len(arr)):
            self.seg[self.size + i] = Node(arr[i])
        for i in range(self.size-1, 0, -1):
            self.seg[i] = merge(self.seg[2*i], self.seg[2*i+1])

    def apply(self, idx, w):
        tmp = self.h - idx.bit_length() + 1
        self.seg[idx] = Node(w * (1<<tmp))
        if idx < self.size: self.lazy[idx] = w

    def push(self, idx):
        if self.lazy[idx] != -INF:
            self.apply(2*idx, self.lazy[idx])
            self.apply(2*idx+1, self.lazy[idx])
            self.lazy[idx] = -INF

    def update(self, l, r, w):
        l += self.size
        r += self.size
        for i in range(self.h, 0, -1):
            if l>>i<<i != 1:
                self.push(l>>i)
            if (r+1)>>i<<i != r+1:
                self.push(r>>i)
        l0, r0 = l, r
        while l <= r:
            if l&1:
                self.apply(l, w)
                l += 1
            if ~r&1:
                self.apply(r, w)
                r -= 1
            l >>= 1; r >>= 1
        l, r = l0, r0
        for i in range(1, self.h+1):
            if l>>i<<i != 1:

```

```

                k = l>>i
                self.seg[k] = merge(self.seg[2*k], self.seg[2*k+1])
            if (r+1)>>i<<i != r+1:
                k = r>>i
                self.seg[k] = merge(self.seg[2*k], self.seg[2*k+1])
        return

    def query(self, l, r):
        l += self.size
        r += self.size
        for i in range(self.h, 0, -1):
            if l>>i<<i != 1:
                self.push(l>>i)
            if (r+1)>>i<<i != r+1:
                self.push(r>>i)
        nl, nr = Node(), Node()
        while l <= r:
            if l&1:
                nl = merge(nl, self.seg[l])
                l += 1
            if ~r&1:
                nr = merge(self.seg[r], nr)
                r -= 1
            l >>= 1; r >>= 1
        return merge(nl, nr)

```

2.3. Trie

2.3.1 1D Trie

```

class Trie:
    def __init__(self, alpha, max_node, func):
        # alpha: the number of characters
        # max_node: the maximum number of nodes
        # func: function that translates characters to index num
        self.alpha = alpha
        self.tries = [0] * (alpha * max_node)
        self.cnt = [0] * max_node
        self.end = [0] * max_node
        self.num = 0
        self.f = func

    def insert(self, word):
        i = 0
        self.cnt[i] += 1
        for ch in word:
            idx = self.f(ch)
            pt = self.alpha * i + idx
            if self.tries[pt] == 0:
                self.num += 1
                self.tries[pt] = self.num
            i = self.tries[pt]
            self.cnt[i] += 1
        self.end[i] += 1

```

3. Graphs

3.1. Shortest Path

3.1.1 Dijkstra

```
import heapq as hq
INF = 10**18

def dijkstra(graph, root):
    n = len(graph)
    dist = [INF]*n
    dist[root] = 0
    heap = []
    hq.heappush(heap, (0, root))
    while heap:
        val, cur = hq.heappop(heap)
        if dist[cur] != val: continue
        for nxt, w in graph[cur]:
            if dist[cur] + w < dist[nxt]:
                dist[nxt] = dist[cur] + w
                hq.heappush(heap, (dist[nxt], nxt))
    return dist
```

3.1.2 Bellman Ford

```
INF = 10**18

def bellman_ford(edges, n, root):
    dist = [INF]*n
    dist[root] = 0
    for i in range(n):
        for j in range(len(edges)):
            if dist[edges[j][0]] < INF and dist[edges[j][1]] > dist[edges[j][0]] +
                ↪ edges[j][2]:
                dist[edges[j][1]] = dist[edges[j][0]] + edges[j][2]
            if i == n-1: return -1
    return dist
```

3.1.3 SPFA

```
from collections import deque
INF = 10**18

def SPFA(graph, root):
    n = len(graph)
    dist = [INF for _ in range(n)]
    dist[root] = 0
    Q = deque([root])
    inque = [0 for _ in range(n)]
    visited = [0 for _ in range(n)]
    visited[root] = 1
    inque[root] = 1
    while Q:
        cur = Q.popleft()
        inque[cur] = 0
        for nxt, weight in graph[cur]:
            if dist[nxt] > dist[cur] + weight:
                dist[nxt] = dist[cur] + weight
```

```
if not inque[nxt]:
    Q.append(nxt)
    inque[nxt] = 1
    visited[nxt] += 1
    if visited[nxt] >= len(graph):
        return -1
```

```
return dist
```

3.1.4 Floyd Warshall

```
INF = 10**18

def floyd(edges, n):
    floyd = [[INF]*n for _ in range(n)]
    for i in range(n+1):
        floyd[i][i] = 0
    for u, v, w in edges:
        floyd[u][v] = min(w, floyd[u][v])
    for k in range(n):
        for i in range(n):
            for j in range(n):
                floyd[i][j] = min(floyd[i][j], floyd[i][k] + floyd[k][j])
    return floyd
```

3.2. SCC and similar

3.2.1 SCC

```
def makescc(graph, n):
    seq, low = [-1]*n, [-1]*n
    onst = [0]*n
    sccs = []
    num = 0
    st1, st2 = [], []
    # st1: tarjan stack, st2: dfs stack

    for start in range(n):
        if seq[start] != -1: continue
        st2.append((start, -1, 0))
        while st2:
            cur, pr, flag = st2.pop()
            if flag:
                for nxt in graph[cur]:
                    if onst[nxt]:
                        low[cur] = min(low[cur], seq[nxt])
            if pr != -1:
                low[pr] = min(low[pr], low[cur])
            if seq[cur] == low[cur]:
                scc = []
                while st1:
                    v = st1.pop()
                    onst[v] = 0
                    scc.append(v)
                    if v == cur: break
                sccs.append(sorted(scc))
            else:
                if seq[cur] != -1: continue
                seq[cur] = low[cur] = num
```

```

    num += 1
    st1.append(cur); onst[cur] = 1
    st2.append((cur, pr, 1))
    for nxt in graph[cur]:
        if seq[nxt] == -1:
            st2.append((nxt, cur, 0))

    return sccs[::-1]

```

3.2.2 2-SAT

```

n, m = map(int, input().split())
graph = [[] for _ in range(2*n+1)]
edge = []
for _ in range(m):
    a, b = map(int, input().split())
    graph[~a].append(b)
    graph[~b].append(a)
    edge.append((-a, b))
    edge.append((-b, a))
sccs = makescc(2*n+1, graph)
group = [-1 for _ in range(2*n+1)]
for i in range(len(sccs)):
    for node in sccs[i]:
        group[node] = i
flag = 1
solution = [0 for _ in range(n)]
for i in range(1, n+1):
    if group[i] == group[-i]:
        flag = 0
        break
    if group[i] > group[-i]:
        solution[i-1] = 1

```

3.2.3 Articulation

```

def makedst(n, graph):
    visited = [0 for _ in range(n)]
    dst = [[] for _ in range(n)]
    parent = [-1 for _ in range(n)]
    for root in range(n):
        if visited[root]: continue
        stack = [(root, -1)]
        while stack:
            node, prt = stack.pop()
            if visited[node]: continue
            if prt != -1:
                dst[node].append(prt)
                dst[prt].append(node)
            visited[node] = 1
            for adj in graph[node]:
                if visited[adj]: continue
                stack.append((adj, node))
    return dst

def artic_point(n, graph):
    cnt = 0
    dst = makedst(n, graph)

```

```

parent = [-1 for _ in range(n)]
points = []
visited = [0 for _ in range(n)]
seq = [0 for _ in range(n)]
minidx = [0 for _ in range(n)]
for root in range(n):
    if visited[root]: continue
    stack = [(root, 0)]
    while stack:
        node, flag = stack.pop()
        if flag:
            if node == root:
                if len(dst[root]) > 1:
                    points.append(node)
                continue
            for adj in dst[node]:
                if parent[node] == adj: continue
                if seq[node] <= minidx[adj]:
                    points.append(node)
                    break
            for adj in graph[node]:
                if parent[node] == adj: continue
                minidx[node] = min(minidx[node], minidx[adj])
        else:
            visited[node] = 1
            seq[node] = cnt
            minidx[node] = cnt
            cnt += 1
            stack.append((node, 1))
            for adj in dst[node]:
                if visited[adj]: continue
                stack.append((adj, 0))
                parent[adj] = node

    return points

```

```

def artic_line(n, graph):
    cnt = 0
    dst = makedst(n, graph)
    parent = [-1 for _ in range(n)]
    lines = []
    visited = [0 for _ in range(n)]
    seq = [0 for _ in range(n)]
    minidx = [0 for _ in range(n)]
    for root in range(n):
        if visited[root]: continue
        stack = [(root, 0)]
        while stack:
            node, flag = stack.pop()
            if flag:
                for adj in dst[node]:
                    if parent[node] == adj: continue
                    if seq[node] < minidx[adj]:
                        if node < adj: lines.append((node, adj))
                        else: lines.append((adj, node))
                for adj in graph[node]:
                    if parent[node] == adj: continue

```

```

        minidx[node] = min(minidx[node], minidx[adj])
    else:
        visited[node] = 1
        seq[node] = cnt
        minidx[node] = cnt
        cnt += 1
        stack.append((node, 1))
        for adj in dst[node]:
            if visited[adj]: continue
            stack.append((adj, 0))
            parent[adj] = node
return lines

```

3.3. Matching & Flow

3.3.1 Bipartite Matching - Hopcroft Karp

```

from collections import deque
INF = 10**9

def bfs(bigraph, matchL, matchR, dist):
    Q = deque()
    dist[0] = INF
    for i in range(1, len(bigraph)):
        if matchL[i] == 0:
            dist[i] = 0
            Q.append(i)
        else:
            dist[i] = INF
    while Q:
        x = Q.popleft()
        if dist[x] < dist[0]:
            for y in bigraph[x]:
                z = matchR[y]
                if dist[z] == INF:
                    dist[z] = dist[x] + 1
                    Q.append(z)
    return dist[0] != INF

def dfs(bigraph, matchL, matchR, dist, work, root):
    st = [root]
    path = []
    while st:
        x = st[-1]
        if x == 0:
            for u, v in zip(st[:-1], path):
                matchL[u] = v
                matchR[v] = u
            return 1
        while work[x] < len(bigraph[x]):
            y = bigraph[x][work[x]]
            work[x] += 1
            z = matchR[y]
            if dist[z] == dist[x] + 1:
                path.append(y)
                st.append(z)
            work[z] = 0

```

```

        break
    else:
        st.pop()
        if path: path.pop()
        dist[x] = INF
    return 0

def hopcroft_karp(bigraph, n, m):
    # 1-base index
    matchL = [0] * (n+1)
    matchR = [0] * (m+1)
    dist = [0] * (n+1)
    ans = 0
    while bfs(bigraph, matchL, matchR, dist):
        work = [0] * (n+1)
        for i in range(1, n+1):
            if matchL[i] == 0:
                ans += dfs(bigraph, matchL, matchR, dist, work, i)
    return ans

```

3.3.2 Dinic Algorithm

```

from collections import deque

def bfs(source, sink):
    level[:] = [-1]*len(graph)
    level[source] = 0
    Q = deque([source])
    while Q:
        cur = Q.popleft()
        for nxt in graph[cur]:
            if level[nxt] == -1 and capa[cur][nxt] - flow[cur][nxt] > 0:
                level[nxt] = level[cur] + 1
                Q.append(nxt)
    return level[sink] != -1

def dfs(source, sink, work):
    st = [(source, 10**9)]
    while st:
        cur, flw = st[-1]
        if cur == sink:
            for _ in range(level[sink]):
                nxt, res = st.pop()
                work[st[-1][0]] -= 1
                flow[st[-1][0]][nxt] += flw
                flow[nxt][st[-1][0]] -= flw
            return flw
        for idx in range(work[cur], len(graph[cur])):
            nxt = graph[cur][idx]
            work[cur] += 1
            res = capa[cur][nxt] - flow[cur][nxt]
            if res > 0 and level[nxt] == level[cur] + 1:
                if flw > res: flw = res
                st.append((nxt, flw))
                break
        else: st.pop()
    return 0

```

```
def dinic(source, sink):
    total = 0
    while bfs(source, sink):
        work = [0 for _ in range(len(graph))]
        while 1:
            minflow = dfs(source, sink, work)
            if minflow == 0: break
            total += minflow
    return total
```

```
n = int(input())
graph = [[] for _ in range(n)]
capa = [[0]*n for _ in range(n)]
flow = [[0]*n for _ in range(n)]
level = [-1 for _ in range(n)]
```

3.3.3 Dinic with low memory

```
from collections import deque
```

```
def bfs(source, sink):
    level[:] = [-1]*len(graph)
    level[source] = 0
    Q = deque([source])
    while Q:
        cur = Q.popleft()
        for nxt, c, _ in graph[cur]:
            if level[nxt] == -1 and c:
                level[nxt] = level[cur] + 1
                Q.append(nxt)
    return level[sink] != -1
```

```
def dfs(source, sink, work):
    st = [(source, -1, 10**9)]
    while st:
        cur, ix, flw = st[-1]
        if cur == sink:
            for _ in range(level[sink]):
                nxt, idx, res = st.pop()
                work[st[-1][0]] -= 1
                graph[st[-1][0]][idx][1] -= flw
                revidx = graph[st[-1][0]][idx][2]
                graph[nxt][revidx][1] += flw
            return flw
        for idx in range(work[cur], len(graph[cur])):
            nxt, res = graph[cur][idx][:2]
            work[cur] += 1
            if res > 0 and level[nxt] == level[cur] + 1:
                if flw > res: flw = res
                st.append((nxt, idx, flw))
                break
            else: st.pop()
    return 0
```

```
def dinic(source, sink):
    total = 0
```

```
while bfs(source, sink):
    work = [0 for _ in range(len(graph))]
    while 1:
        minflow = dfs(source, sink, work)
        if minflow == 0: break
        total += minflow
    return total
```

```
n, m = map(int, input().split())
graph = [[] for _ in range(n)]
level = [-1 for _ in range(n)]
for _ in range(m):
    a, b = map(int, input().split())
    a -= 1; b -= 1
    # connected node, capa, edge num
    graph[a].append([b, 1, len(graph[b])])
    graph[b].append([a, 0, len(graph[a])-1])
print(dinic(0, n-1))
```

3.3.4 MCMF

```
from collections import deque
INF = 10**9
```

```
def spfa(source, sink, graph, capa, flow, cost):
    path = [-1] * len(graph)
    dist = [INF] * len(graph)
    inque = [0] * len(graph)
    dist[source] = 0
    inque[source] = 1
    Q = deque([source])
    while Q:
        cur = Q.popleft()
        inque[cur] = 0
        for nxt in graph[cur]:
            if capa[cur][nxt] > flow[cur][nxt] and dist[nxt] > dist[cur] +
                ↪ cost[cur][nxt]:
                dist[nxt] = dist[cur] + cost[cur][nxt]
                path[nxt] = cur
                if not inque[nxt]:
                    inque[nxt] = 1
                    Q.append(nxt)
    return path
```

```
def mcmf(source, sink, graph, capa, flow, cost):
    size = len(graph)
    ans = [0, 0] # max flow, min cost
    while 1:
        path = spfa(source, sink, graph, capa, flow, cost)
        if path[sink] == -1: break
        aug = INF
        j = sink
        while j != source:
            i = path[j]
            aug = min(aug, capa[i][j] - flow[i][j])
            j = i
        j = sink
```

```

while j != source:
    i = path[j]
    flow[i][j] += aug
    flow[j][i] -= aug
    ans[1] += cost[i][j] * aug
    j = i
ans[0] += aug
return ans

```

3.4. Trees

3.4.1 LCA

```

def lca(u, v):
    # 1-base index
    if level[u] > level[v]: u, v = v, u
    for j in range(len(parent[0])-1, -1, -1):
        if level[parent[v][j]] >= level[u]:
            v = parent[v][j]
    if u == v: return u
    for j in range(len(parent[0])-1, -1, -1):
        if parent[u][j] == parent[v][j]: continue
        u, v = parent[u][j], parent[v][j]
    return parent[u][0]

```

```

n = int(input())
graph = [[] for _ in range(n+1)]
for _ in range(n-1):
    u, v = map(int, input().split())
    graph[u].append(v)
    graph[v].append(u)
parent = [[0]*n.bit_length() for _ in range(n+1)]
st = [1]
while st:
    node = st.pop()
    for child in graph[node]:
        if parent[node][0] == child: continue
        st.append(child)
        parent[child][0] = node
for j in range(1, len(parent[0])):
    for i in range(1, n+1):
        parent[i][j] = parent[parent[i][j-1]][j-1]

```

3.4.2 HLD

```

def buildhld(graph, n, root=1):
    # 1-base index
    parent = [0]*(n+1)
    depth = [0]*(n+1)
    size = [0]*(n+1)
    heavy = [-1]*(n+1)

    st = [(root, 0)]
    while st:
        node, flag = st.pop()
        if flag:
            hchild = -1; maxsize = 0
            size[node] = 1

```

```

        for child in graph[node]:
            if parent[node] == child: continue
            size[node] += size[child]
            if size[child] > maxsize:
                maxsize = size[child]
                hchild = child
            heavy[node] = hchild
        else:
            st.append((node, 1))
            for child in graph[node]:
                if parent[node] == child: continue
                parent[child] = node
                depth[child] = depth[node] + 1
                st.append((child, 0))

```

```

head = [0]*(n+1)
pos = [0]*(n+1)

# decomposition
st = [root] # chain head
num = 0
while st:
    h = st.pop()
    cur = h
    while cur != -1:
        head[cur] = h
        pos[cur] = num
        num += 1
        for nxt in graph[cur]:
            if parent[cur] == nxt or heavy[cur] == nxt:
                continue
            st.append(nxt)
            cur = heavy[cur]

```

```

return parent, depth, size, heavy, head, pos

```

```

def update_path(u, v, parent, depth, head, pos, a, b):
    while head[u] != head[v]:
        if depth[head[u]] < depth[head[v]]:
            u, v = v, u
        seg.update(pos[head[u]], pos[u], a, b)
        u = parent[head[u]]
    if depth[u] > depth[v]:
        u, v = v, u
    # include lca (u == lca)
    seg.update(pos[u], pos[v], a, b)
    return

```

```

def getsum_path(u, v, parent, depth, head, pos):
    ans = 0
    while head[u] != head[v]:
        if depth[head[u]] < depth[head[v]]:
            u, v = v, u
        ans = (ans + seg.range_sum(pos[head[u]], pos[u])) & mask
        u = parent[head[u]]
    if depth[u] > depth[v]:

```

```

    u, v = v, u
    # include lca (u == lca)
    ans = (ans + seg.range_sum(pos[u], pos[v])) & mask
    return ans

```

4. DP

4.1. Basic DP

4.1.1 LIS

```

from bisect import bisect_left
INF = 10**18

n = int(input())
arr = [*map(int, input().split())]
leng = [-INF]
dp = [0]*n
for i in range(n):
    dp[i] = bisect_left(leng, arr[i])
    if dp[i] == len(leng):
        leng.append(arr[i])
    else:
        leng[dp[i]] = arr[i]
tmp = max(dp)
print(tmp)
lis = []
for i in range(n-1, -1, -1):
    if tmp == dp[i]:
        tmp -= 1
        lis.append(arr[i])
print(*lis[::-1])

```

4.2. DP optimization

4.2.1 Convex Hull Trick

$dp[i] = \min_{j < i} (dp[j] + A[j] * B[i])$ 형태의 관계식을 $A[j] = a, B[i] = x, dp[j] = b$ 로 하여 $y = ax + b$ 꼴의 직선들의 볼록 꺾질을 만들어 해결한다. $A[j]$ 의 단조성이 보장되어야 한다.

```

def search(val, st):
    left, right = 0, len(st)-1
    while left + 1 < right:
        mid = (left + right) // 2
        t1 = st[mid][0] * val + st[mid][1]
        t2 = st[mid+1][0] * val + st[mid+1][1]
        if t1 < t2: right = mid
        else: left = mid+1
    t1 = st[left][0] * val + st[left][1]
    t2 = st[right][0] * val + st[right][1]
    return left if t1 < t2 else right

def check(l1, l2, l3):
    return (l3[1]-l2[1]) * (l1[0]-l2[0]) <= (l2[1]-l1[1]) * (l2[0]-l3[0])

n = int(input())
a = [*map(int, input().split())]
b = [*map(int, input().split())]

```

```

dp = [0]*n
st = []
st.append((b[0], 0))
for i in range(1, n):
    idx = search(a[i], st)
    dp[i] = st[idx][0] * a[i] + st[idx][1]
    while len(st) > 1 and check(st[-2], st[-1], (b[i], dp[i])):
        st.pop()
    st.append((b[i], dp[i]))
print(dp[-1])

```

5. String

5.1. Pattern Matching

5.1.1 KMP

```

def makefail(s):
    failf = [0] * len(s)
    j = 0
    for i in range(1, len(s)):
        while j > 0 and s[i] != s[j]:
            j = failf[j-1]
        if s[i] == s[j]:
            j += 1
        failf[i] = j
    return failf

def kmp_search(s1, s2):
    # find s2 in s1
    failf = makefail(s2)
    result = []
    j = 0
    for i in range(len(s1)):
        while j > 0 and s1[i] != s2[j]:
            j = failf[j-1]
        if s1[i] == s2[j]:
            j += 1
            if j == len(s2):
                result.append(i-j+2)
                j = failf[j-1]
    return result

```

5.1.2 Aho Corasick

```

from collections import deque

```

```

class Aho_Corasick:
    def __init__(self, alpha, func, patterns):
        self.alpha = alpha
        self.f = func
        self.patterns = patterns

        max_node = sum(len(p) for p in patterns) + 1
        self.trie = [0] * (alpha * max_node)
        self.fail = [0] * max_node
        self.out = [-1] * max_node
        self.end = [0] * max_node

```

```

self.num = 0
self.occnt = [0] * max_node # count the occurrence
self.out_link = [-1] * max_node # linked list
self.items = []

for pid, pattern in enumerate(patterns):
    self.insert(pattern, pid)
self.build()

def insert(self, word, pid):
    i = 0
    for ch in word:
        idx = self.f(ch)
        pos = self.alpha * i + idx
        if self.trie[pos] == 0:
            self.num += 1
            self.trie[pos] = self.num
            i = self.trie[pos]
        self.end[i] += 1
    prev = self.out[i]
    self.items.append((pid, len(word), prev))
    self.out[i] = len(self.items) - 1

def build(self):
    q = deque()
    for c in range(self.alpha):
        v = self.trie[c]
        if v != 0:
            self.fail[v] = 0
            q.append(v)
    self.occnt[0] = self.end[0]
    self.out_link[0] = -1

    while q:
        cur = q.popleft()
        fcur = self.fail[cur]
        self.occnt[cur] = self.end[cur] + self.occnt[fcur]
        self.out_link[cur] = fcur if self.out[fcur] != -1 else self.out_link[fcur]
        for c in range(self.alpha):
            pos = self.alpha * cur + c
            nxt = self.trie[pos]
            if nxt != 0:
                self.fail[nxt] = self.trie[self.alpha * fcur + c]
                q.append(nxt)
            else:
                self.trie[pos] = self.trie[self.alpha * fcur + c]

def findall(self, text):
    ans = []
    i = 0
    for pos, ch in enumerate(text):
        idx = self.f(ch)
        i = self.trie[self.alpha * i + idx]
        node = i
        while node != -1:

```

```

            head = self.out[node]
            while head != -1:
                pid, leng, nxt = self.items[head]
                ans.append((pos-leng+1, pos, pid))
                head = nxt
            node = self.out_link[node]
    return ans

```

5.2. Substring Algorithm

5.2.1 Rolling Hash

```

def hashing(left, right):
    return (hashlst[right] - powlst[right-left+1] * hashlst[left-1]) % mod

mod = 10**18 + 9; p = 31
s = input().rstrip()
hashlst = [0 for _ in range(len(s)+1)]
powlst = [1 for _ in range(len(s)+1)]
for i in range(1, len(s)+1):
    powlst[i] = powlst[i-1] * p % mod
hashlst[0] = ord(s[0]) - 97
for i in range(1, len(s)):
    hashlst[i] = (hashlst[i-1]*p + ord(s[i])-97)%mod

```

5.2.2 Z

```

def makez(s):
    n = len(s)
    l, r = 0, 0
    z = [0]*n; z[0] = n
    for i in range(1, n):
        if i <= r:
            z[i] = min(r-i+1, z[i-1])
            while i+z[i] < n and s[i+z[i]] == s[z[i]]:
                z[i] += 1
            if r < i+z[i]-1:
                l, r = i, i+z[i]-1
    return z

```

5.2.3 Manacher

```

def manacher(s):
    n = len(s)
    r, ctr = 0, 0
    arr = [0]*n
    for i in range(n):
        if i <= r:
            arr[i] = min(arr[2*ctr-i], r-i)
            while i-arr[i] >= 1 and i+arr[i] < n-1 and s[i-arr[i]-1] == s[i+arr[i]+1]:
                arr[i] += 1
            if r < i+arr[i]:
                r, ctr = i+arr[i], i
    return arr

```

5.2.4 Suffix array & LCP array

```

from bisect import bisect_left

```

```
def count_sort(arr, key, k):
    # count sorting by key[idx], range: 0 - k
    n = len(arr)
    cnt = [0] * (k+1)
    for val in arr:
        cnt[key[val]] += 1
    pfs = [0] * (k+1)
    for i in range(k):
        pfs[i+1] = pfs[i] + cnt[i]
    out = [0] * n
    for val in arr:
        out[pfs[key[val]]] = val
        pfs[key[val]] += 1
    return out

def make_suffix(s):
    n = len(s)
    if n == 0: return []
    sa = [*range(n)]
    uniq = []
    for ch in sorted(s):
        if not uniq or ch != uniq[-1]:
            uniq.append(ch)
    rank = [bisect_left(uniq, ch) for ch in s]
    k = 1
    while k < n:
        key1 = [r+1 for r in rank]
        key2 = [rank[i+k]+1 if i+k < n else 0 for i in range(n)]
        sa = count_sort(sa, key2, max(key2))
        sa = count_sort(sa, key1, max(key1))
        cnt = 0
        new_rank = [0] * n
        new_rank[sa[0]] = 0
        for i in range(1, n):
            a, b = sa[i], sa[i-1]
            ra, rb = rank[a], rank[b]
            rka = rank[a+k] if a+k < n else -1
            rkb = rank[b+k] if b+k < n else -1
            if ra != rb or rka != rkb:
                cnt += 1
            new_rank[a] = cnt
        rank = new_rank[:]
        if cnt == n-1: break
        k *= 2
    return sa

def makelcp(s, sa):
    n = len(s)
    if n <= 1: return []
    rev = [0] * n
    for i, p in enumerate(sa):
        rev[p] = i
    lcp = [0] * (n-1)
    prvl = 0
    for i in range(n):
        r = rev[i]
```

```
        if r == 0: prvl = 0; continue
        j = sa[r-1]
        while i+prvl < n and j+prvl < n and s[i+prvl] == s[j+prvl]:
            prvl += 1
        lcp[r-1] = prvl
        if prvl: prvl -= 1
    return lcp
```

6. Misc

6.1. Useful Codes

6.1.1 Fast I/O

```
def main():
    import io, os
    input = io.BufferedReader(io.FileIO(0), 1<<18).readline
    out = []
    for _ in range(int(input())):
        a, b = map(int, input().split())
        out.append(f"{a+b}")
    os.write(1, '\n'.join(out).encode())
    os._exit(0)
main()
```

6.1.2 Coordinate Compression

```
def compress(lst, typ=0):
    # typ == 1: return trace list together
    new = [(lst[i], i) for i in range(len(lst))]
    new.sort()
    ans = [0 for _ in range(len(lst))]
    trace = []
    k = 0
    for i in range(len(lst)):
        if i > 0 and new[i][0] > new[i-1][0]:
            k += 1
        ans[new[i][1]] = k
        if trace and trace[-1] == new[i][0]: continue
        trace.append(new[i][0])
    if typ: return ans, trace
    return ans
```

6.2. Others

6.2.1 Graph Theorem

코닉의 정리: 이분 그래프에서 (최대 매칭) = (최소 정점 커버).

이분 그래프에서 (정점 수) = (최대 매칭) + (최대 독립집합 크기).

홀의 결혼 정리: 이분 그래프에서 좌측 집합 L 의 모든 부분집합 $S \subset L$ 에 대해 $|N(S)| \geq |S|$ 이면 L 에 대한 완전 매칭이 존재한다.

6.2.2 Useful Number

NTT Friendly Prime.

$998244353 = 119 \times 2^{23} + 1$, primitive root = 3.

$985661441 = 235 \times 2^{22} + 1$, primitive root = 3.

Useful prime numbers.

$10^8 + 7$, $10^9 + 7$, $10^9 + 9$, $10^{11} + 3$, $10^{12} + 39$, $10^{18} + 3$.

6.2.3 Useful Sequences

Fibonacci F_n .

$$F_0 = 0, F_1 = 1, F_n = F_{n-1} + F_{n-2}.$$

0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377, 610, 987, 1597, 2584, $\dots$.Lucas L_n .

$$L_0 = 2, L_1 = 1, L_n = L_{n-1} + L_{n-2}.$$

2, 1, 3, 4, 7, 11, 18, 29, 47, 76, 123, 199, 322, 521, 843, 1364, 2207, $\dots$.Tribonacci T_n .

$$T_0 = 0, T_1 = 0, T_2 = 1, T_n = T_{n-1} + T_{n-2} + T_{n-3}.$$

0, 0, 1, 1, 2, 4, 7, 13, 24, 44, 81, 149, 274, 504, 927, 1705, $\dots$.Catalan C_n .

$$C_0 = 1, C_n = \frac{1}{n+1} \binom{2n}{n}.$$

1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900, $\dots$.Central Binomial Coefficient B_n .

$$B_n = \binom{2n}{n}.$$

1, 2, 6, 20, 70, 252, 924, 3432, 12870, 48620, 184756, 705432, 2704156, $\dots$.Partition Number $p(n)$.

$$p(0) = 1, p(n) = \sum_{k \neq 0} (-1)^{k-1} p! \left(n - \frac{k(3k-1)}{2} \right).$$

1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42, 56, 77, 101, 135, 176, 231, $\dots$.Bell B_n .1부터 n 까지 자연수로 이루어진 집합을 분할하는 방법의 수.

$$B_0 = 1, B_{n+1} = \sum_{k=0}^n \binom{n}{k} B_k.$$

1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975, 678570, $\dots$.Partition number $p(n, k)$.정수 n 을 정확히 k 개의 양의 정수의 합으로 나타내는 방법의 수이다 (순서는 무시).

$$p(0, 0) = 1, p(n, 0) = 0 \ (n > 0), p(0, k) = 0 \ (k > 0).$$

$$p(n, k) = p(n-1, k-1) + p(n-k, k).$$

Stirling (1st kind) $s(n, k)$. x^n 에서의 x^k 계수이다.

$$s(0, 0) = 1, s(n, 0) = 0 \ (n > 0), s(n, k) = s(n-1, k-1) - (n-1)s(n-1, k).$$

Stirling (2nd kind) $S(n, k)$.1부터 n 까지의 자연수로 이루어진 집합을 k 개로 분할하는 방법의 수이다.

$$S(0, 0) = 1, S(n, 0) = 0 \ (n > 0), S(n, k) = S(n-1, k-1) + kS(n-1, k).$$